Kopilotti Sales - product overview

Kopilotti Sales digitizes used-vehicle price negotiation. Natural-language interaction and the commercial decision are separated: an LLM may support the conversation, while a deterministic server-side policy engine makes the commercial decision within boundaries defined by the dealer.

1. Product purpose

Vehicle information and other purchasing steps can be online while price negotiation still requires a manual exchange. Kopilotti Sales turns that step into a controlled digital flow without transferring commercial authority away from the dealer.

The dealer defines the policy. The system applies it consistently. Cases outside the automated boundary are escalated for human review.

2. Customer journey

1. The customer opens a vehicle-specific negotiation flow.
2. The customer submits an offer.
3. The server loads the vehicle and the applicable commercial policy.
4. The policy engine returns `ACCEPT`, `COUNTER`, `REJECT`, or `ESCALATE`.
5. An accepted outcome may reserve the vehicle in the same database transaction as the decision, session transition, and audit event.
6. The customer continues in the dealer's own completion process, or a person handles an escalated case.

Kopilotti Sales limits its role to digital price negotiation and a controlled continuation path. The dealer remains responsible for its own transaction, payment, and handover processes.

3. Commercial boundaries

The dealer defines the commercial policy, including the price floor, target price, list price, round limits, and escalation rules. Kopilotti Sales applies the policy; it does not invent a pricing strategy.

A stale policy, missing vehicle, mismatch between the vehicle and policy, or policy conflict leads to escalation instead of a guessed decision. A person retains authority over exceptional cases.

4. Deterministic policy engine

The policy engine returns one of four states: accept, counter, reject, or escalate. The price floor bounds acceptance and counteroffers. The same validated input and policy produce the same commercial decision.

The commercial decision is made by a server-side policy engine isolated from the LLM. This boundary reduces the impact of LLM manipulation but is not presented as complete protection against every attack.

5. Reservation and double-booking prevention

The PostgreSQL model permits only one active reservation for the same dealer and vehicle through a database-enforced unique index. The accepted decision, reservation, session transition, and audit event share a transaction. If one step fails, the unit of work is rolled back.

Persistence and reservation mechanisms are implemented and tested but have not been production-verified in this review.

6. Persistence and application audit history

Negotiation and purchase sessions, idempotency records, decisions, and audit events have PostgreSQL persistence paths. The application audit history is ordered per dealer and hash-chained. The application path is append-only, and database triggers reject updates and deletions from the audit-event table.

This describes application and database behavior in the repository. It does not claim an immutable external ledger, event-sourced architecture, or production verification.

7. DDN and verifiability

DDN (Deterministic Decision Network) preparation includes deterministic canonicalization, domain-separated hashes, a bounded submission adapter, and a safe local boundary for constructing and displaying a receipt link for a locally verified result.

A hash supports integrity checking, but a hash alone does not authenticate the receipt issuer.

Live DDN verification, signer authentication, quorum and trust profiles, public anchoring, and an independent offline verifier are roadmap items. The current implementation does not present them as complete production capabilities.

8. Security boundaries

- The LLM may support conversation but does not decide the price or commercial outcome.
- LLM isolation reduces the impact of manipulation but is not a promise of complete prompt-injection protection.
- A hash shows integrity-related consistency, not issuer identity or a trust chain.
- Roadmap capabilities are not prerequisites for the current decision flow.
- This overview does not authorize deployment, migrations, or access to production services.

9. Production-readiness status

Status: **not production-verified**.

Readiness and schema-verification gates are implemented and tested. Approved RTO/RPO targets, verified backup and restore, named operational owners, and production alert routing remain separate decision gates.

The publicly documented current scope and production boundaries are available in the Kopilotti Sales public README.

10. Implemented and tested

- digital price negotiation
- a server-side commercial decision isolated from the LLM
- deterministic `ACCEPT`, `COUNTER`, `REJECT`, and `ESCALATE` logic
- dealer-defined commercial boundaries and price-floor enforcement
- deterministic canonicalization and hash generation
- a safe local boundary for constructing and displaying a receipt link

11. Not production-verified

- atomic vehicle reservation
- database-enforced prevention of concurrent active reservations
- PostgreSQL persistence for negotiation and purchase sessions
- application audit history and hash chain
- append-only application path for audit events
- readiness and schema-verification gates

Persistence and reservation mechanisms are implemented and tested but have not been production-verified in this review.

12. Roadmap and research directions

- live DDN verification
- signer authentication and pinned trust profiles
- quorum verification and public anchoring
- an independent offline verifier
- byte-exact runtime replay
- a signed committed decision artifact
- proof-gated execution
- zero-knowledge proofs
- approved RTO/RPO targets
- verified backup and restore
- named operational owners and response times

These are target or research directions, not current capabilities.

13. Demo and further information

- Open the demo
- Suomenkielinen tuote-esittely
- Suomenkielinen A4-PDF
- Repository README
- License

The demo illustrates the product flow. It does not prove production verification for capabilities marked as not production-verified.